

Autonomous Drone-Based Human Detection Using Raspberry Pi 5

Mahija Ibad Pradipta
Dept. of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
mahijapradipta86@gmail.com

Arief Kurniawan
Dept. of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
arifku@ee.its.ac.id

Eko Mulyanto Yuniarno
Dept. of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
ekomulyanto@its.ac.id

Abstract—The advancement of Unmanned Aerial Vehicle (UAV) technology has created new opportunities for Search and Rescue (SAR) operations. In Indonesia, where natural disasters occur frequently, there is an urgent need for systems capable of assisting victim detection efficiently in areas that are difficult for humans to access. This study designs and implements an autonomous system on a Holybro S500 drone capable of detecting disaster victims automatically using a Raspberry Pi 5 and a USB webcam. The system integrates a Pixhawk 6C Flight Controller as the main navigation controller and a Raspberry Pi 5 as a companion computer responsible for human detection using a fine-tuned YOLOv8n model and autonomous control through MAVSDK Python via the MAVLink protocol. The research encompasses electrical design with a UBEC 5A-based power regulation for stable edge computing, mechanical design ensuring aerodynamic stability through center-of-gravity optimization, and a multi-threaded software architecture integrating GStreamer-based video acquisition, ONNX-accelerated deep learning inference, and an event-driven state machine for autonomous navigation. Fine-tuning the YOLOv8n model on the COCO2017 person class achieved a precision of 0.809, recall of 0.646, and mAP@50 of 0.757. Field validation demonstrated stable real-time detection at 15 FPS on the Raspberry Pi 5 with the ONNX 320×320 deployment configuration, confirming the system’s viability for automated SAR missions.

Index Terms—Autonomous Drone, Search and Rescue, Raspberry Pi 5, YOLOv8, MAVSDK, Edge Computing

I. INTRODUCTION

The advancement of Unmanned Aerial Vehicle (UAV) technology has opened significant opportunities for utilization in various fields, including Search and Rescue (SAR) operations. Drones possess the capability to reach areas that are difficult for humans to access, such as mountain slopes, dense forests, or disaster-affected areas with damaged infrastructure, thereby increasing the speed, efficiency, and effectiveness of rescue operations in emergency situations [1]. The global search and rescue drone market continues to exhibit substantial growth, reflecting increasing recognition of UAV potential in humanitarian operations [2]. Modern UAVs, particularly rotary-wing quadcopters, offer unique advantages for SAR applications due to their ability to hover in place and perform flexible maneuvers, enabling stable visual observation over specific areas of interest [8].

In Indonesia, the need for autonomous drone systems in disaster contexts is increasingly urgent. Based on data from

the National Disaster Management Agency (BNPB) in 2024, there were 3,472 disaster events resulting in 540 fatalities. Although the frequency of disasters decreased compared to the previous year, the fatality and damage rates increased by almost 96%, indicating the limited early detection and rapid victim search systems deployed in the field [3]. Despite this urgency, adoption of drone technology in humanitarian operations within developing countries remains limited due to technical, regulatory, and institutional barriers [4].

Existing autonomous SAR drone systems face several critical challenges that limit their operational effectiveness. First, computational limitations of older single-board computers (e.g., Raspberry Pi 3B+ and 4) result in high inference latency and low frame rates that are insufficient for real-time autonomous decision-making [5], [6]. Second, power management for edge computing devices on drones remains a largely unaddressed engineering challenge, with voltage instability during peak AI workloads causing system failures. Third, most existing systems rely on general-purpose object detection models without task-specific optimization for aerial human detection scenarios.

This research focuses on designing and implementing an autonomous drone system capable of automatically detecting disaster victims. The system is designed so that the drone can operate without manual control, autonomously navigate to sweep the affected area, detect the presence of humans through visual imagery, and make decisions such as returning to a safe point when a victim has been detected or the battery is critically low. The scope of this work is bounded by the following constraints: (1) single-UAV operation using the Holybro S500 frame with Pixhawk 6C flight controller; (2) edge computing limited to a Raspberry Pi 5 (8GB) in headless Linux configuration; (3) visual detection restricted to RGB camera without gimbal stabilization, targeting only the human class (single-class detection); (4) testing conducted under daylight outdoor conditions with non-fully-occluded targets; and (5) flight controller communication via MAVSDK Python over the MAVLink protocol.

The contributions of this work include: (1) a complete hardware-software co-design for autonomous SAR on a low-cost quadcopter platform, (2) an iterative electrical design methodology that resolves edge-computing power stability

challenges on the Raspberry Pi 5, (3) a fine-tuned single-class YOLOv8n model optimized for aerial human detection on resource-constrained hardware, and (4) a multi-threaded software architecture integrating GStreamer video acquisition, ONNX-accelerated inference, and an event-driven state machine with real-time web-based monitoring.

II. RELATED WORK

A. Autonomous Search and Rescue Drone

Elashaal et al. [5] developed an autonomous SAR drone system using the Holybro S500 platform with a Pixhawk 2.4.8 flight controller and Raspberry Pi 4. The system employed the YOLOv4-tiny algorithm for real-time human detection, achieving an mAP of 96.44% on top-view imagery. However, the computational limitations of the Raspberry Pi 4 caused high latency and low frame rates in complex terrain conditions. Furthermore, the flight control integration lacked a comprehensive autonomous state machine, relying instead on pre-programmed waypoint missions without adaptive behavior based on detection results. Field testing was also limited to specific environments such as desert and coastal areas.

B. Real-Time Human Detection for SAR

Jayalath and Munasinghe [6] integrated TensorFlow Fast R-CNN-based human detection with autonomous navigation on a Raspberry Pi 4 using the Okutama-Action dataset. The drone was capable of calculating GPS coordinates of victims based on bounding box positions. Although detection accuracy reached 84.1%, the processing time of 8 seconds per frame was deemed insufficient for dynamic autonomous missions requiring real-time control loop feedback. The system was also highly dependent on natural lighting conditions, and the two-stage detection architecture (R-CNN) inherently limits throughput on resource-constrained edge devices.

C. Target Detection and Avoidance for UAVs

Varatharasan et al. [7] focused on low-cost hardware efficiency using a Raspberry Pi 3B+ and Intel Movidius NCS2. Their research highlighted the use of MAVSDK API for Offboard control mode, demonstrating the viability of velocity-based setpoint commands for autonomous drone navigation. The advantage lay in system stability on low-power devices; however, depth estimation accuracy remained limited due to reliance on stadiametric rangefinding approaches, and the system required an external neural compute stick rather than performing inference natively on the companion computer.

Table I summarizes the comparative analysis of related work against the proposed system.

Compared to existing work, our system leverages the significantly more powerful Raspberry Pi 5 (ARM Cortex-A76, 8GB RAM), employs a fine-tuned YOLOv8n model exported to ONNX format for optimized native edge inference without external accelerators, and implements a complete multi-threaded autonomous architecture with GStreamer video pipeline, event-driven state machine, and real-time web-based monitoring.

TABLE I
COMPARISON WITH EXISTING SAR DRONE SYSTEMS

Study	Computer	Model	FPS	Autonomy
Elashaal [5]	RPi 4	YOLOv4-	Low	Waypoint
Jayalath [6]	RPi 4	tiny Fast R-CNN	0.125	GPS-based
Varatha. [7]	RPi 3B+ +NCS2	Custom	Med	Offboard
Ours	RPi 5	YOLOv8n	15	State Machine

III. THEORETICAL BACKGROUND

A. YOLOv8 Architecture

YOLOv8 is a single-stage object detection algorithm based on Convolutional Neural Networks (CNNs) that performs detection and classification in a single forward pass [11]. The eighth generation introduces an anchor-free detection head and a C2f (Cross Stage Partial with two convolutions) module that improves accuracy without sacrificing inference speed. The YOLOv8n (nano) variant, containing approximately 3.2 million parameters, is specifically designed for deployment on edge devices. When exported to the ONNX (Open Neural Network Exchange) format, the model benefits from runtime optimizations including graph fusion and memory layout optimization, enabling efficient inference through OpenCV's DNN backend without requiring GPU acceleration.

B. MAVLink and MAVSDK

MAVLink (Micro Air Vehicle Link) is a lightweight binary messaging protocol designed for communication between flight controllers and external systems [12]. The protocol supports bidirectional exchange of telemetry data, command messages, and mission items with minimal bandwidth overhead. MAVSDK provides a high-level software development kit with Python API bindings that abstract the low-level MAVLink message handling, enabling developers to issue asynchronous navigation commands such as Offboard velocity setpoints (VelocityNedYaw), mission uploads, and telemetry subscriptions [13]. The combination allows autonomous control logic running on a companion computer to seamlessly command the flight controller while continuously receiving sensor feedback.

C. PX4 Autopilot and Offboard Mode

PX4 is an open-source autopilot firmware that manages the flight stack, including EKF2 (Extended Kalman Filter) sensor fusion for state estimation and PID-based attitude and rate controllers for motor output [16]. The Offboard flight mode enables external companion computers to command the drone's velocity, position, or attitude setpoints at high frequency. PX4 enforces safety constraints by rejecting external commands that violate geofence boundaries or flight envelope limits, and automatically reverting to a failsafe mode (Return-to-Launch or Land) if the companion computer communication stream is interrupted.

IV. SYSTEM DESIGN AND ARCHITECTURE

The system methodology is structured into three main stages: Drone Control System Behavior Planning, System Planning and Design, and System Integration and Implementation. Fig. 1 illustrates the high-level system architecture.

A. Hardware Platform

The drone platform utilized is the Holybro S500 quadcopter, a research-grade X-frame constructed from carbon fiber and ABS plastic, supporting a payload capacity of up to 1.5 kg [9]. Table II summarizes the key hardware components. The flight controller is a Pixhawk 6C running PX4 Autopilot firmware (v1.14), which manages flight stack operations including EKF2 sensor fusion for position estimation and brushless motor speed control [16]. A Raspberry Pi 5 (8GB RAM) serves as the companion computer, featuring an ARM Cortex-A76 processor and VideoCore VII GPU capable of running Edge AI workloads such as local deep learning inference [10]. However, the Raspberry Pi 5 has a sensitive power profile (5V/5A), requiring a dedicated voltage regulation system to avoid booting failures or voltage sag under peak computational load. A Logitech C922 webcam connected via USB 3.0 provides visual data acquisition at 640×480 resolution and 30 FPS for the detection pipeline [14]. A Skydroid T10 remote controller operating on the 2.4 GHz frequency via SBUS protocol serves as the master safety override switch, holding the highest authority to activate OFFBOARD mode or trigger emergency interruption during flight [18].

TABLE II
HARDWARE COMPONENT SPECIFICATIONS

Component	Specification
Drone Frame	Holybro S500 X-frame (carbon fiber/ABS)
Flight Controller	Pixhawk 6C, PX4 v1.14 firmware
Companion Computer	Raspberry Pi 5, 8GB RAM, ARM Cortex-A76
Camera	Logitech C922, USB 3.0, 640×480@30FPS
Battery (Main)	LiPo 4S 14.8V
Power Regulator	UBEC Hobbywing 5A (step-down to 5V)
RC Controller	Skydroid T10, 2.4 GHz, SBUS

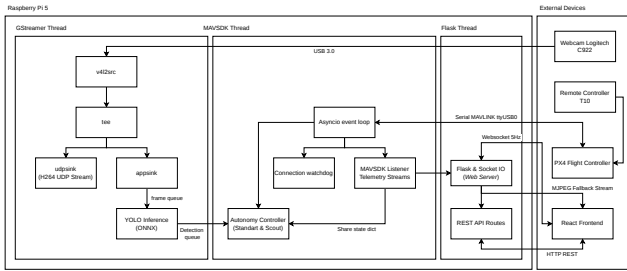


Fig. 1. Overall system architecture depicting the multi-rate integration between flight subsystem (Pixhawk 6C/PX4), autonomous computation subsystem (Raspberry Pi 5), and real-time monitoring subsystem (React frontend via SocketIO).

B. Electrical Design

The electrical design underwent iterative development to achieve stable computational operations during flight.

1) *Initial Design and Failure Analysis*: The initial power design utilized 18650 Li-ion batteries (2S configuration) with XL4015E1/XL4016E1 step-down modules. However, the continuous discharge rate of these batteries could not accommodate transient current spikes during YOLOv8 inference initialization, triggering voltage sag and boot failures. A critical incident occurred when power was supplied through the GPIO pins, which lack OVP/OCP circuits, causing permanent PMIC damage.

2) *Final UBEC-Based Design*: The final architecture employs a Hobbywing UBEC 5A step-down module powered by the main LiPo 4S (14.8V) battery [15]. Power is distributed via an XT60 Y-Splitter: one branch to the PDB for ESC/motors and Pixhawk, the other to the UBEC 5A for the Raspberry Pi 5 and webcam. Fig. 2 illustrates the parallel battery configuration. The data interface between the Raspberry Pi 5 and Pixhawk 6C uses a direct USB connection for MAVLink communication [12], [13].

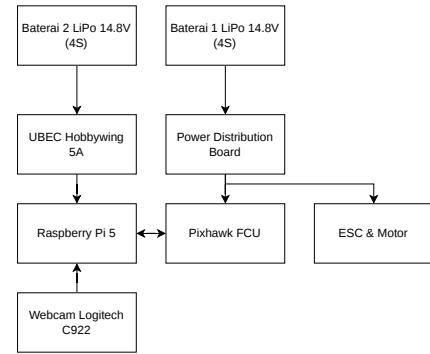


Fig. 2. Parallel battery power distribution schematic. The LiPo 4S battery supplies both the PDB (propulsion) and UBEC 5A (companion computer) via an XT60 Y-Splitter.

C. Mechanical Design

The mechanical design integrates additional components onto the Holybro S500 frame while maintaining balance through center-of-gravity (CG) optimization. All components are positioned centrally and symmetrically on the lower frame section, following the pendulum stability principle. Seven modular 3D-printed PLA components were designed using CAD software. Table III lists the mechanical fastener specifications.

D. Software Architecture

The software architecture integrates three concurrent subsystems operating as a multi-rate system on the Raspberry Pi 5, synchronized through message queue communication patterns to prevent race conditions.

TABLE III
MECHANICAL FASTENER SPECIFICATIONS

Fastener	Size	Application
Structural bolt	M3×15mm	Main frame attachment
Compartment bolt	M2×8mm	Protective enclosures
Module bolt	M2×12mm	Mount and shield fixing
Extension bolt	M2×15mm	Extended compartments
Lock nut	M2	All mechanical joints

1) *Visual-to-Control Pipeline*: The primary data flow handles visual perception and autonomous navigation decisions. The GStreamer thread acquires raw video from the Logitech C922 via `v4l2src`, applies pixel reduction through a tee element, and distributes frames to both a UDP H.264 stream (`udpsink`) and the inference pipeline (`appsink` to frame queue). The YOLO consumer thread retrieves frames from the queue, executes deep learning inference using the ONNX-exported YOLOv8n model at 320×320 resolution, and produces bounding box detections that are placed into a detection queue. The Autonomy Controller consumes detection events, processes them through the current state machine context, and generates VelocityNedYaw navigation vectors transmitted via MAVSDK to the PX4 flight controller for motor actuation.

2) *Telemetry-to-Frontend Pipeline*: The MAVSDK Listener subscribes to PX4 telemetry (IMU, GPS, battery) via MAVLink and updates a shared state dictionary. A Flask & SocketIO server samples this dictionary at 5 Hz and transmits JSON payloads over WebSocket to a React-based dashboard, operating on a read-only principle to ensure zero interference with the control loop.

E. Autonomous Behavior Design

The autonomous Scout Mission consists of four phases: (1) System Initialization validating all subsystem readiness, (2) Manual Takeoff followed by pilot-initiated OFFBOARD activation via Skydroid T10, (3) Scout Execution Cycle managed by the event-driven state machine, and (4) mission completion through RTL or failsafe termination.

1) *State Machine for Victim Search*: The core search behavior is implemented as an event-driven state machine with four states:

SCAN: The drone performs slow yaw rotation at a hover position to expand the camera's observation area. The human detection module operates continuously during this phase.

APPROACH: Upon confirmed human detection with sufficient confidence, the drone navigates toward the target based on the object's position relative to the camera frame center. Movement is performed incrementally to maintain flight stability.

DOCUMENT: When the target is within documentation range, the system captures visual evidence of the detected victim.

RETURN: After documentation, the drone returns to the initial search position and re-enters the SCAN state to continue searching for additional targets.

Fig. 3 illustrates the complete state machine transition diagram for the scout mission.

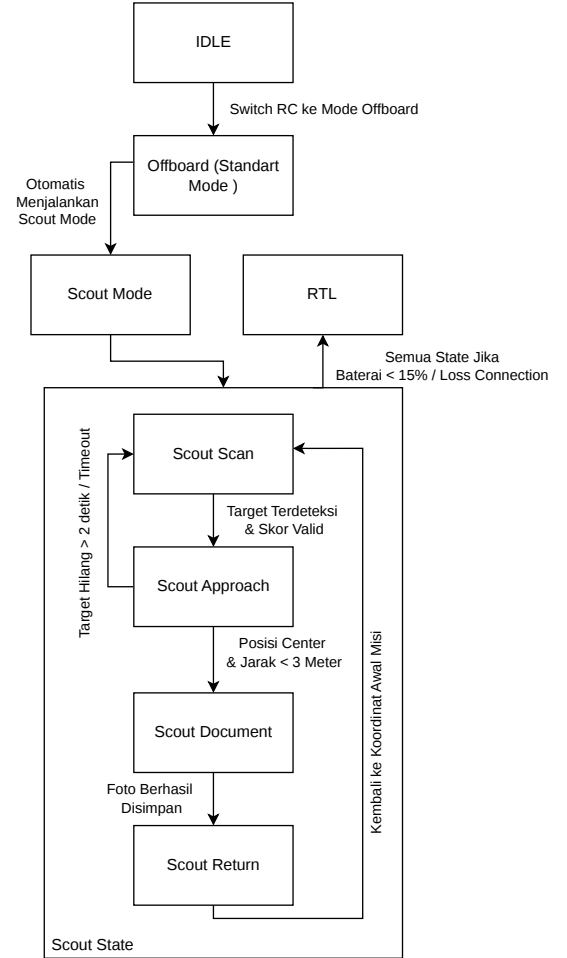


Fig. 3. State machine diagram for the autonomous scout mission, showing transitions between SCAN, APPROACH, DOCUMENT, and RETURN states with failsafe conditions.

2) *Target Validation and Lock*: The system implements a persistence check that validates detection consistency across multiple consecutive frames before generating a confirmed event. Target lock logic evaluates bounding box area (45% weight), confidence score (25%), proximity to frame center (20%), and inter-frame spatial consistency (10%). The system maintains focus on a locked target unless a new candidate scores 35% higher, preventing unstable target switching.

3) *Failsafe Mechanisms*: Automatic failsafe responses handle low battery (<15%), telemetry loss, and communication failures by triggering RTL or emergency landing. RC override via the Skydroid T10 allows the pilot to assume manual control at any time.

V. SYSTEM IMPLEMENTATION

A. Hardware Integration

The Holybro S500 was assembled with custom integration of the Pixhawk 6C flight controller as the base platform. The Raspberry Pi 5 was mounted using 3D-printed PLA enclosures positioned centrally on the lower frame section to maintain the center of gravity. The Logitech C922 webcam was attached via an adjustable downward-facing mount connected through USB 3.0. XT60 connectors were soldered onto the UBEC 5A module with Y-Splitter for plug-and-play LiPo connectivity. Fig. 4 shows the fully assembled system.

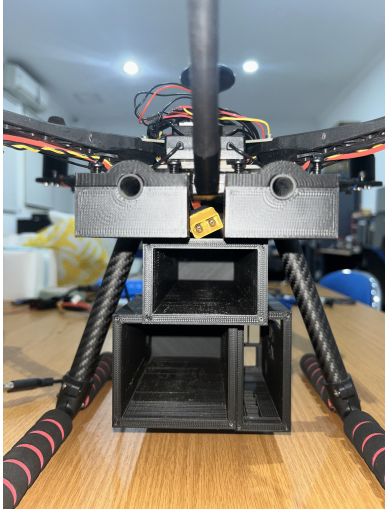


Fig. 4. Fully assembled Holybro S500 drone with integrated 3D-printed compartments housing the Raspberry Pi 5, UBEC 5A, LiPo battery, and downward-facing camera mount.

B. Software Implementation

The backend system was developed as a Python-based multi-threaded application running on Raspberry Pi OS configured in full headless mode (SSH-only, no X11/Wayland desktop environment). This configuration was adopted to minimize RAM consumption and eliminate display server overhead that competes with the inference engine for computational resources. The system architecture consists of three primary threads operating concurrently:

GStreamer Thread: Acquires video via `v4l2src`, splits the stream using a `tee` element into a UDP H.264 branch (`udpsink`) and a YOLO inference branch (`appsink` to frame queue). Camera parameters are locked at initialization via `v4l2-ctl` [17].

MAVSDK Thread: Runs a Python `asyncio` event loop maintaining MAVLink communication with the Pixhawk 6C. The MAVSDK Listener subscribes to telemetry streams and updates a shared state dictionary. The Autonomy Controller consumes detection events and issues `VelocityNedYaw` set-points for navigation control.

Flask Thread: Hosts a REST API with SocketIO WebSocket for the monitoring dashboard, sampling shared state at

5 Hz and providing an MJPEG fallback stream for bandwidth-constrained links.

The YOLOv8n model was fine-tuned using Ultralytics, exported to ONNX, and loaded with OpenCV's DNN module at 320×320 resolution. The YOLO consumer implements a non-blocking queue pattern, always processing the most recent frame. The frontend was built using Vite and React with WebSocket-based reconnection, displaying telemetry, camera feed, detection overlays, and state machine logs.

VI. TESTING AND EVALUATION

Testing was conducted in both indoor laboratory environments for software integration and outdoor open-field areas for flight validation with simulated human targets at 2–5 meter distances. The evaluation strategy was structured progressively: first validating hardware reliability (power supply stability), then optimizing the operating environment, followed by benchmarking the detection module, and finally conducting integrated field validation.

A. Power Supply Validation

The power subsystem underwent multiple design iterations to achieve stable edge-computing operations, as summarized in Table IV.

TABLE IV
POWER SUPPLY DESIGN ITERATIONS

Method	Status	Result	Failure Mode
Mini UPS GPIO 3A	Boot	Fatal	PMIC damage from transient spike via unprotected GPIO
XL4015E1 5A	Active	Unstable	Under-voltage warning (0x5000) and periodic reboots
XL4016E1 8A	Idle OK	Fail under load	Voltage sag from Li-ion low C-Rating
UBEC 5A + LiPo	Full load	Stable	Final configuration

The initial UPS 18650-based design failed when YOLOv8 inference caused a transient spike exceeding the battery's discharge rating. The GPIO power path lacked surge protection (OVP/OCV), resulting in permanent PMIC damage. After migrating to the UBEC 5A via USB-C from the LiPo 4S battery, voltage stability was confirmed at 5V throughout all phases including peak inference workloads, completely eliminating the instability.

B. Operating System Optimization

Stripping the desktop environment (X11/Wayland) and operating in full headless mode reduced idle RAM consumption, eliminated display server context switches, and improved inference latency consistency. Camera parameter tuning via `v4l2-ctl` (exposure locking, auto-white balance deactivation) proved more reliable in headless mode, preventing the detection flickering observed during desktop-mode testing.

C. Inference Resolution Benchmarking

Experiments compared YOLOv8n base and YOLOv8n-pose variants at multiple input resolutions (256×256 and 320×320) on the Raspberry Pi 5 using a standardized benchmarking framework. The benchmarking system utilized the Ultralytics library with ONNX Runtime backend, with parameters locked at: webcam input resolution 320×240 , single-class detection (person, confidence threshold 0.4), and metrics averaged over two 6-second test rounds. Post fine-tuning, the ONNX-exported model was benchmarked across three resolutions, as shown in Table V.

TABLE V
ONNX INFERENCE BENCHMARK ON RASPBERRY PI 5

Resolution	FPS	Latency (ms)
256×256	35.54	28.13
320×320	23.72	42.17
640×640	5.84	171.22

Although the 256×256 resolution offered higher FPS throughput, qualitative evaluation revealed significant feature degradation on small human objects, particularly at the operating altitude range where targets occupy relatively few pixels. The 640×640 resolution produced excessively high latency (171.22 ms) unsuitable for real-time autonomous control. Given that spatial detail is critical for preventing false negatives in aerial detection, the 320×320 resolution was selected as it maintained real-time performance (>15 FPS) while preserving superior detection sharpness for the target distance range of 2–5 meters.

D. YOLOv8n Fine-Tuning Results

The YOLOv8n model was fine-tuned on the COCO2017 dataset filtered exclusively for the person class (ID 0), comprising 244,799 instances [11]. The training configuration is detailed in Table VI.

TABLE VI
YOLOv8N FINE-TUNING CONFIGURATION

Parameter	Value
Base Model	YOLOv8n (Pretrained COCO)
Epochs	50
Image Size (Training)	640×640
Image Size (Deployment)	320×320
Batch Size	128
Optimizer	SGD
Initial Learning Rate	0.002
Mosaic Augmentation	0.5 (Off at last 10 epochs)
HSV Augmentation	Disabled
Mixed Precision (AMP)	Enabled

Training at 640×640 resolution ensured the model learned fine-grained features of small objects before downscaling to 320×320 for deployment. HSV augmentation was disabled as aerial human detection relies more on contour features than color variations. After 50 epochs, the model achieved

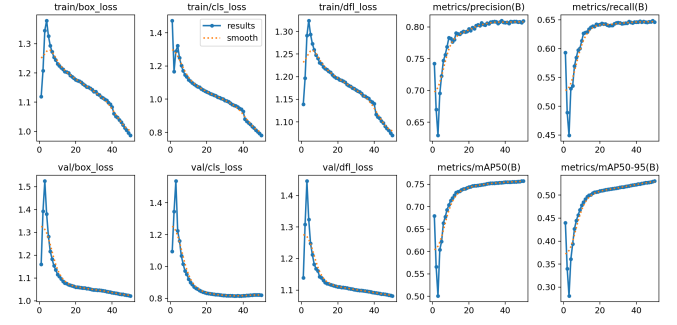


Fig. 5. Training and validation curves over 50 epochs showing stable convergence of box/cls/dfl losses and progressive improvement of precision, recall, mAP@50, and mAP@50-95 metrics.

TABLE VII
FINE-TUNING VALIDATION METRICS

Metric	Value
Precision	0.809
Recall	0.646
mAP@50	0.757
mAP@50-95	0.531
Maximum F1-Score	0.72
Optimal Confidence Threshold	0.224

optimal convergence as shown in Fig. 5. Validation metrics are presented in Table VII.

The precision of 0.809 minimizes the risk of pursuing non-human objects. The recall of 0.646 appears conservative due to small objects in COCO; practical detection rates are higher at closer distances (2–5 meters). The mAP@50 of 0.757 surpasses the pretrained model, validating that single-class fine-tuning improves detection accuracy.

E. Field Validation

Final validation was conducted through outdoor field testing to observe dynamic system behavior under realistic conditions with human targets positioned at distances of 2–5 meters from the drone’s operating altitude. The testing environment consisted of an open field area with natural background elements (vegetation, ground textures, shadows) to evaluate the system’s robustness against visual noise sources typical of disaster environments.

The evaluation yielded the following findings across three critical performance dimensions:

Detection Stability: Bounding boxes maintained stable target lock during yaw rotation and approach maneuvers. The persistence check effectively filtered transient false detections from visual artifacts (vegetation, shadows), ensuring only validated events reached the state machine. Target lock maintained focus during the APPROACH phase despite bounding box position shifts from drone movement.

Computational Efficiency: The ONNX YOLOv8n at 320×320 sustained ~ 15 FPS, providing a ~ 67 ms control loop update period sufficient for smooth VelocityNedYaw

navigation. The GStreamer pipeline maintained stable frame delivery without buffer overflows.

False Positive Resilience: The confidence threshold of 0.224, from F1-Score curve analysis, delivered rapid detection without excessive false positives. The single-class fine-tuning concentrated discriminative capacity on human features, and the combination with persistence check proved a robust strategy balancing sensitivity against specificity.

The integrated system comprising headless OS optimization, UBEK 5A power regulation, GStreamer video pipeline, and the fine-tuned YOLOv8n model successfully met the technical specifications required for an autonomous drone victim search mission system. Fig. 6 shows the React-based monitoring dashboard during an active detection session, displaying real-time telemetry, YOLO bounding box overlays, and the storage gallery of documented scout photos.

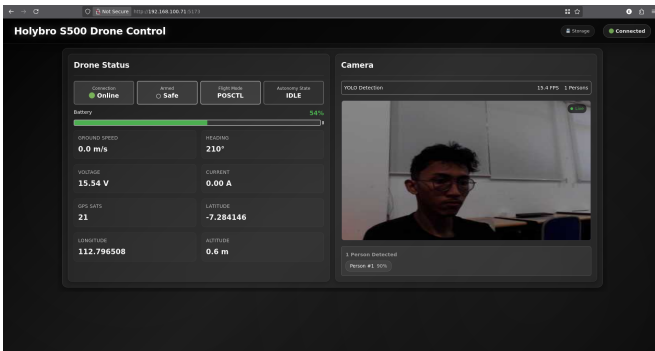


Fig. 6. Real-time monitoring dashboard showing drone telemetry (battery, GPS, altitude), live camera feed with YOLOv8n bounding box overlay (person detected at 90% confidence, 15.4 FPS), and storage gallery of documented scout mission photos.

F. Discussion

The Raspberry Pi 5 represents a meaningful generational improvement over the Raspberry Pi 4 for edge AI drone applications. While Elashaal et al. [5] reported low frame rates on RPi 4 with YOLOv4-tiny, our system achieves 15 FPS with YOLOv8n, enabling closed-loop autonomous control. The iterative electrical design revealed that the power delivery interface (GPIO vs. USB-C) is as critical as the power source itself GPIO lacks transient protection, making it unsuitable for AI inference workloads. The single-class fine-tuning strategy, training at 640×640 before deploying at 320×320 , proved highly effective for concentrating model capacity on aerial human detection.

VII. CONCLUSION AND FUTURE WORK

The fine-tuned YOLOv8n model achieved a precision of 0.809, recall of 0.646, and mAP@50 of 0.757 on the person class, while maintaining 15 FPS inference throughput on the Raspberry Pi 5 at 320×320 resolution. Field validation confirmed the system's ability to perform stable detection, target tracking through the persistence check mechanism, and autonomous navigation decisions under realistic outdoor

conditions. The monitoring subsystem provided operators with real-time situational awareness through a React-based dashboard accessible over Wi-Fi or LTE connectivity.

The current implementation has several limitations defining future work directions: extending to multi-drone collaborative operations, incorporating infrared/thermal sensors for night operations, adding gimbal stabilization, and conducting testing in actual disaster scenarios. Future research will also explore lightweight architectures beyond YOLOv8n, GPS-stamped victim coordinate reporting for SAR command systems, and edge-optimized runtimes such as TensorRT Lite.

REFERENCES

- [1] M. Lyu, Y. Zhao, C. Huang, and H. Huang, "Unmanned aerial vehicles for search and rescue: A survey," *Remote Sensing*, vol. 15, no. 13, p. 3266, 2023.
- [2] Fact.MR, "Search and rescue drone market study," 2024. [Online]. Available: <https://www.factmr.com/report/search-and-rescue-drone-market>
- [3] Badan Nasional Penanggulangan Bencana, *Data Bencana Indonesia 2024*. Jakarta, Indonesia: Pusat Data, Informasi, dan Komunikasi Kebencanaan BNPB, 2025.
- [4] B. M. Sopha, A. M. S. Asih, and J. I. Agriawan, "Adopters and non-adopters of drones in humanitarian operations: An empirical evidence from a developing country," *Progress in Disaster Science*, vol. 21, p. 100314, 2024.
- [5] Z. A. Elashaal, Y. H. Lamin, M. K. Elfandi, and A. A. Eluheshi, "Autonomous search and rescue drone," *Libyan Journal of Informatics*, vol. 1, no. 1, pp. 1–8, 2024.
- [6] K. Jayalath and S. R. Munasinghe, "Drone-based autonomous human identification for search and rescue missions in real-time," in *Proc. 10th Int. Conf. Information and Automation for Sustainability (ICIAfS)*, Moratuwa, Sri Lanka, 2021.
- [7] V. Varatharasan, A. S. S. Rao, E. Toutounji, J.-H. Hong, and H.-S. Shin, "Target detection, tracking and avoidance system for low-cost UAVs using AI-based approaches," in *Proc. Int. Workshop Research, Education and Development on Unmanned Aerial Systems (RED-UAS)*, Cranfield, UK, 2019, pp. 142–149.
- [8] R. Austin, *Unmanned Aircraft Systems: UAV Design, Development and Deployment*. John Wiley & Sons, 2021.
- [9] Holybro Robotics, *S500 Quadcopter Frame Technical Manual*. Holybro, 2023.
- [10] Raspberry Pi Foundation, *Raspberry Pi 5 Technical Overview*. Raspberry Pi Ltd., 2024.
- [11] H. Zhou, J. Wang, Z. Liu, and Y. Li, "SOD-YOLOv8: Enhancing YOLOv8 for small object detection," *Sensors*, vol. 24, no. 19, p. 6209, 2024.
- [12] MAVLink Development Team, *Micro Air Vehicle Link Protocol Specification*. MAVLink.io, 2024.
- [13] MAVSDK Project, "MAVSDK — A MAVLink SDK for drones, cameras and ground systems," <https://mavsdk.mavlink.io/>, 2025.
- [14] A. Kumar, S. Chaudhary, S. Sangal, and R. Dhama, "Review on computer vision using OpenCV," *Int. J. Computer Applications Technology and Research*, vol. 3, no. 12, pp. 756–761, 2024.
- [15] D. McNulty, "A review of Li-ion batteries for autonomous mobile robots," *Sensors and Actuators A: Physical*, vol. 339, p. 113567, 2022.
- [16] S. D'Angelo, F. Pagano, F. Longobardi, F. Ruggiero, and V. Lippiello, "Efficient development of model-based controllers in PX4 firmware: A template-based customization approach," *Control Engineering Practice*, vol. 140, p. 105529, 2024.
- [17] J. Song, H. Jeong, and J. Jeong, "Performance optimization of object tracking algorithms in OpenCV on GPUs," *Applied Sciences*, vol. 12, no. 15, p. 7801, 2022.
- [18] STM Research Group, "Effective RF transmitter and receiver system using 2.4 GHz for unmanned aerial vehicles," *Communications and Aerospace Research Trends*, vol. 7, pp. 15367–15377, 2024.